Module 1: Face Detection and Landmark-Based Alignment

Project	Face Verification Pipeline -- AgeDB-30 / LFW
Notebook	`notebooks/01_detect_align.ipynb`
Date	2026-03-16
Dependency	EDA results from `notebooks/EDA.ipynb`
Input	`data/{agedb,lfw}/imgs/*.jpg` (pre-cropped 112x112)
Output	`data/aligned/{agedb,lfw}/*.jpg`, `logs/detection_log_*.csv`, updated `results/pairs_*.csv`

Objective

This module performs landmark-based face alignment on all images referenced by the verification pair protocol. The goal is to normalize face geometry to the ArcFace reference template (5-point similarity transform), which is a prerequisite for all downstream embedding extraction and verification experiments.

Key findings from EDA that motivate this step:

- Eye-line tilt across both datasets has a mean of ~19 degrees with non-zero variance, indicating residual misalignment in the original pre-crops.
- Without alignment, EER degrades by 10--14 percentage points (per ablation studies reported in the project analysis).
- The `2d106det` landmark regressor achieves 100% detection rate on both datasets, making it the optimal choice over scene-level detectors (RetinaFace achieves <1% detection on these 112x112 crops without padding).

1. Configuration and Imports

```
import os
import sys
import csv
import time
import hashlib
import warnings
from pathlib import Path
from copy import deepcopy

import cv2
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import FancyArrowPatch
from tqdm.auto import tqdm

from insightface.model_zoo import get_model
from insightface.utils.face_align import norm_crop, estimate_norm,
arcface_dst

warnings.filterwarnings("ignore", category=FutureWarning)
plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update({
    "figure.dpi": 120,
    "savefig.dpi": 150,
    "font.size": 10,
    "axes.titlesize": 11,
    "axes.labelsize": 10,
    "figure.facecolor": "white",
})

SEED = 42
np.random.seed(SEED)

PROJECT_ROOT = Path("..")
DATA_DIR     = PROJECT_ROOT / "data"
RESULTS_DIR  = PROJECT_ROOT / "results"
LOGS_DIR     = PROJECT_ROOT / "logs"
OUTPUT_DIR   = PROJECT_ROOT / "notebooks" / "align_output"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

DATASETS = {
    "agedb": DATA_DIR / "agedb",
    "lfw":   DATA_DIR / "lfw",
}

ALIGNED_DIRS = {
    name: DATA_DIR / "aligned" / name for name in DATASETS
}
for d in ALIGNED_DIRS.values():
    d.mkdir(parents=True, exist_ok=True)

IMAGE_SIZE = 112
LMK106_5PT_INDICES = [38, 88, 86, 52, 61]

ARCFACE_TEMPLATE = arcface_dst.copy()

print(f"Project root : {PROJECT_ROOT.resolve()}")
print(f"Datasets     : {list(DATASETS.keys())}")
print(f"Image size   : {IMAGE_SIZE}x{IMAGE_SIZE}")
```

```
Project root : /Users/jakub/Desktop/NASK_ZADANIE
Datasets     : ['agedb', 'lfw']
Image size   : 112x112
```

2. Methodology

2.1 Alignment Pipeline

The alignment procedure consists of three stages:

1. **Landmark regression** -- The `2d106det` model (ONNX, 192x192 input) regresses 106 facial landmarks from a bounding-box crop. Since our images are already pre-cropped faces, we pass the full image as the bounding box.

2. **5-point extraction** -- From the 106 landmarks, we extract 5 canonical points: left eye center (index 38), right eye center (88), nose tip (86), left mouth corner (52), right mouth corner (61).

3. **Similarity transform** -- A 2D similarity transformation (rotation, uniform scale, translation) is estimated to map the detected 5 points onto the ArcFace reference template. The image is then warped using `cv2.warpAffine`.

2.2 Transformation Model

The similarity transform $M \in R^{2\times 3}$ is estimated by minimizing the least-squares error between source landmarks $\{s_i\}_{i=1}^{5}$ and the reference template $\{d_i\}_{i=1}^{5}$:

$$M^{¿} = \arg\min_{M} \sum_{i=1}^{5} \| M \cdot \tilde{s}_i - d_i \|^2$$

where $\tilde{s}_i = [s_{i,x}, s_{i,y}, 1]^{\mathrm{T}}$ denotes homogeneous coordinates. The transform preserves aspect ratio (isotropic scaling) and handles rotation, which corrects for head tilt.

2.3 ArcFace Reference Template

The target landmark positions for a 112x112 output are:

```
template_df = pd.DataFrame(
    ARCFACE_TEMPLATE,
    columns=["x", "y"],
    index=["left_eye", "right_eye", "nose", "mouth_left",
"mouth_right"],
)
template_df.index.name = "landmark"
print("ArcFace reference template (112x112):")
print(template_df.to_string())
```

```
ArcFace reference template (112x112):
                     x          y
landmark
left_eye     38.294601  51.696301
right_eye    73.531799  51.501400
nose         56.025200  71.736603
mouth_left   41.549301  92.365501
mouth_right  70.729897  92.204102
```

2.4 Model Initialization

```
LMK_MODEL_PATH = os.path.expanduser(
    "~/.insightface/models/buffalo_l/2d106det.onnx"
)
assert os.path.exists(LMK_MODEL_PATH), f"Model not found:
{LMK_MODEL_PATH}"

lmk_model = get_model(LMK_MODEL_PATH,
providers=["CPUExecutionProvider"])
lmk_model.prepare(ctx_id=-1)

print(f"Model        : 2d106det")
print(f"Path         : {LMK_MODEL_PATH}")
print(f"Input size   : {lmk_model.input_size}")
print(f"Output       : 106 landmarks (2D)")
print(f"Provider     : CPUExecutionProvider")
```

```
Applied providers: ['CPUExecutionProvider'], with options:
{'CPUExecutionProvider': {}}
Model        : 2d106det
Path         :
/Users/jakub/.insightface/models/buffalo_l/2d106det.onnx
Input size   : (192, 192)
Output       : 106 landmarks (2D)
Provider     : CPUExecutionProvider
```

3. MD5 Deduplication

EDA established that AgeDB contains 55.9% exact duplicates (MD5) and LFW 35.8%. To avoid redundant computation, we identify unique files first and process only those. Duplicate indices receive a copy of the aligned result from their corresponding original.

```
def load_img_list(dataset_dir: Path) -> list[str]:
    """Load img.list and return list of absolute paths indexed by line
number."""
    img_list_path = dataset_dir / "img.list"
    paths = []
    with open(img_list_path, encoding="utf-8") as f:
        for line in f:
```

```
            rel = line.strip()
            if rel:
                paths.append(str(PROJECT_ROOT / rel))
    return paths


def compute_md5(filepath: str) -> str:
    with open(filepath, "rb") as f:
        return hashlib.md5(f.read()).hexdigest()


def build_dedup_map(img_paths: list[str]) -> dict:
    """
    Returns dict with:
      - 'md5_to_first_idx': {md5_hash: first_index}
      - 'idx_to_md5': {index: md5_hash}
      - 'unique_indices': list of indices that are first occurrences
      - 'duplicate_indices': list of (dup_idx, original_idx) pairs
    """
    md5_to_first_idx = {}
    idx_to_md5 = {}
    unique_indices = []
    duplicate_pairs = []

    for idx, path in enumerate(img_paths):
        md5 = compute_md5(path)
        idx_to_md5[idx] = md5
        if md5 not in md5_to_first_idx:
            md5_to_first_idx[md5] = idx
            unique_indices.append(idx)
        else:
            duplicate_pairs.append((idx, md5_to_first_idx[md5]))

    return {
        "md5_to_first_idx": md5_to_first_idx,
        "idx_to_md5": idx_to_md5,
        "unique_indices": unique_indices,
        "duplicate_pairs": duplicate_pairs,
    }

img_lists = {}
dedup_maps = {}

for name, ds_dir in DATASETS.items():
    img_lists[name] = load_img_list(ds_dir)
    dedup_maps[name] = build_dedup_map(img_lists[name])

dedup_summary = pd.DataFrame([
    {
        "Dataset": name,
```

```
        "Total images": len(img_lists[name]),
        "Unique (MD5)": len(dm["unique_indices"]),
        "Duplicates": len(dm["duplicate_pairs"]),
        "Duplicate %": f"{len(dm['duplicate_pairs']) /
len(img_lists[name]) * 100:.1f}%",
    }
    for name, dm in dedup_maps.items()
])

print("Table 1. MD5 Deduplication Summary")
print(dedup_summary.to_string(index=False))
print(f"\nTotal unique images to process: "
      f"{sum(len(dm['unique_indices']) for dm in
dedup_maps.values())}")
print(f"Duplicates to copy:             "
      f"{sum(len(dm['duplicate_pairs']) for dm in
dedup_maps.values())}")
```

```
Table 1. MD5 Deduplication Summary
Dataset  Total images  Unique (MD5)  Duplicates Duplicate %
  agedb         12000          5296        6704       55.9%
    lfw         12000          7701        4299       35.8%

Total unique images to process: 12997
Duplicates to copy:             11003
```

4. Landmark Extraction and Alignment

```
class _FaceProxy:
    """Minimal proxy satisfying the insightface Face interface for
2d106det."""
    def __init__(self, bbox: np.ndarray):
        self.bbox = bbox
    def __setitem__(self, key, value):
        setattr(self, key, value)
    def __getitem__(self, key):
        return getattr(self, key)


def extract_landmarks_106(img: np.ndarray) -> np.ndarray:
    """Extract 106 landmarks from a pre-cropped face image."""
    h, w = img.shape[:2]
    face = _FaceProxy(np.array([0, 0, w - 1, h - 1],
dtype=np.float32))
    lmks = lmk_model.get(img, face)
    return lmks


def get_5pt_from_106(lmks_106: np.ndarray) -> np.ndarray:
    """Extract canonical 5 points from 106-landmark array."""
```

```
    return lmks_106[LMK106_5PT_INDICES].astype(np.float32)


def compute_eye_metrics(kps5: np.ndarray) -> dict:
    """Compute IPD and eye-line tilt from 5-point landmarks."""
    le, re = kps5[0], kps5[1]
    dx, dy = re[0] - le[0], re[1] - le[1]
    ipd = np.sqrt(dx**2 + dy**2)
    tilt_deg = np.degrees(np.arctan2(dy, dx))
    return {"ipd": ipd, "tilt_deg": tilt_deg}

def process_dataset(name: str, img_paths: list[str], dedup: dict) ->
pd.DataFrame:
    """
    Process all images for one dataset:
      1. Extract landmarks for unique images
      2. Compute alignment transform and save aligned crops
      3. Copy results for duplicates

    Returns a DataFrame with per-image detection and alignment
metrics.
    """
    aligned_dir = ALIGNED_DIRS[name]
    records = []

    # Storage for aligned images of unique files (to copy for
duplicates)
    unique_aligned_cache = {}
    unique_kps_cache = {}

    t_start = time.time()

    # --- Phase 1: Process unique images ---
    print(f"[{name}] Phase 1: Processing
{len(dedup['unique_indices'])} unique images...")
    for idx in tqdm(dedup["unique_indices"], desc=f"{name} (unique)"):
        path = img_paths[idx]
        img_name = os.path.basename(path)
        img = cv2.imread(path)

        rec = {
            "img_idx": idx,
            "img_name": img_name,
            "face_detected": False,
            "confidence": 0.0,
            "bbox_x": 0, "bbox_y": 0, "bbox_w": IMAGE_SIZE, "bbox_h":
IMAGE_SIZE,
            "left_eye_x": np.nan, "left_eye_y": np.nan,
            "right_eye_x": np.nan, "right_eye_y": np.nan,
            "nose_x": np.nan, "nose_y": np.nan,
```

```
            "mouth_l_x": np.nan, "mouth_l_y": np.nan,
            "mouth_r_x": np.nan, "mouth_r_y": np.nan,
            "ipd_before": np.nan, "tilt_before": np.nan,
            "ipd_after": np.nan, "tilt_after": np.nan,
            "is_duplicate": False,
            "error": "",
        }

        if img is None:
            rec["error"] = "imread returned None"
            records.append(rec)
            continue

        try:
            lmks106 = extract_landmarks_106(img)
            kps5 = get_5pt_from_106(lmks106)

            metrics_before = compute_eye_metrics(kps5)
            rec["ipd_before"] = metrics_before["ipd"]
            rec["tilt_before"] = metrics_before["tilt_deg"]

            for i, lbl in enumerate(["left_eye", "right_eye", "nose",
"mouth_l", "mouth_r"]):
                rec[f"{lbl}_x"] = float(kps5[i, 0])
                rec[f"{lbl}_y"] = float(kps5[i, 1])

            aligned = norm_crop(img, kps5, image_size=IMAGE_SIZE)

            # Measure alignment quality on the output
            lmks106_after = extract_landmarks_106(aligned)
            kps5_after = get_5pt_from_106(lmks106_after)
            metrics_after = compute_eye_metrics(kps5_after)
            rec["ipd_after"] = metrics_after["ipd"]
            rec["tilt_after"] = metrics_after["tilt_deg"]

            out_path = aligned_dir / img_name
            cv2.imwrite(str(out_path), aligned)

            rec["face_detected"] = True
            rec["confidence"] = 1.0

            unique_aligned_cache[idx] = aligned
            unique_kps_cache[idx] = kps5

        except Exception as e:
            rec["error"] = str(e)
            # Fallback: save original image as-is
            out_path = aligned_dir / img_name
            if img is not None:
                cv2.imwrite(str(out_path), img)
```

```
        records.append(rec)

    # --- Phase 2: Copy duplicates ---
    print(f"[{name}] Phase 2: Copying {len(dedup['duplicate_pairs'])}
duplicates...")
    for dup_idx, orig_idx in tqdm(dedup["duplicate_pairs"],
desc=f"{name} (dups)"):
        dup_name = os.path.basename(img_paths[dup_idx])
        orig_name = os.path.basename(img_paths[orig_idx])

        src = aligned_dir / orig_name
        dst = aligned_dir / dup_name

        if src.exists():
            import shutil
            shutil.copy2(str(src), str(dst))

        # Inherit record from original
        orig_rec = next((r for r in records if r["img_idx"] ==
orig_idx), None)
        if orig_rec:
            rec = orig_rec.copy()
            rec["img_idx"] = dup_idx
            rec["img_name"] = dup_name
            rec["is_duplicate"] = True
        else:
            rec = {
                "img_idx": dup_idx, "img_name": dup_name,
                "face_detected": True, "confidence": 1.0,
                "is_duplicate": True, "error": "",
            }
        records.append(rec)

    elapsed = time.time() - t_start
    print(f"[{name}] Done in {elapsed:.1f}s "
          f"({len(dedup['unique_indices'])} unique +
{len(dedup['duplicate_pairs'])} dups)")

    df =
pd.DataFrame(records).sort_values("img_idx").reset_index(drop=True)
    return df

detection_logs = {}

for name in DATASETS:
    print(f"\n{'='*60}")
    print(f"  Dataset: {name.upper()}")
    print(f"{'='*60}")
    detection_logs[name] = process_dataset(
```

```
        name, img_lists[name], dedup_maps[name]
    )


============================================================
  Dataset: AGEDB
============================================================
[agedb] Phase 1: Processing 5296 unique images...

{"model_id":"ddd695e98a1643acb7671226d5e14a7b","version_major":2,"version_minor":0}

[agedb] Phase 2: Copying 6704 duplicates...

{"model_id":"993eb942e7404a888be2ab79eb0eb9fe","version_major":2,"version_minor":0}

[agedb] Done in 19.4s (5296 unique + 6704 dups)

============================================================
  Dataset: LFW
============================================================
[lfw] Phase 1: Processing 7701 unique images...

{"model_id":"1e006a9b480943b6b85b0a6e49d7b37d","version_major":2,"version_minor":0}

[lfw] Phase 2: Copying 4299 duplicates...

{"model_id":"8b32e8cd044d45b695812ce9b89be73f","version_major":2,"version_minor":0}

[lfw] Done in 29.6s (7701 unique + 4299 dups)

print("Table 2. Detection and Alignment Summary\n")
for name, df in detection_logs.items():
    n_total = len(df)
    n_detected = df["face_detected"].sum()
    n_failed = n_total - n_detected
    n_dup = df["is_duplicate"].sum()
    rate = n_detected / n_total * 100
    print(f"  {name.upper()}:")
    print(f"    Total images      : {n_total}")
    print(f"    Detected          : {n_detected} ({rate:.2f}%)")
    print(f"    Failed            : {n_failed}")
    print(f"    Duplicates copied : {n_dup}")
    if n_failed > 0:
        failed = df[~df["face_detected"]]
        print(f"    Failed indices    : {failed['img_idx'].tolist()[:20]}")
    print()
```

```
Table 2. Detection and Alignment Summary

  AGEDB:
    Total images      : 12000
    Detected          : 12000 (100.00%)
    Failed            : 0
    Duplicates copied : 6704

  LFW:
    Total images      : 12000
    Detected          : 12000 (100.00%)
    Failed            : 0
    Duplicates copied : 4299
```

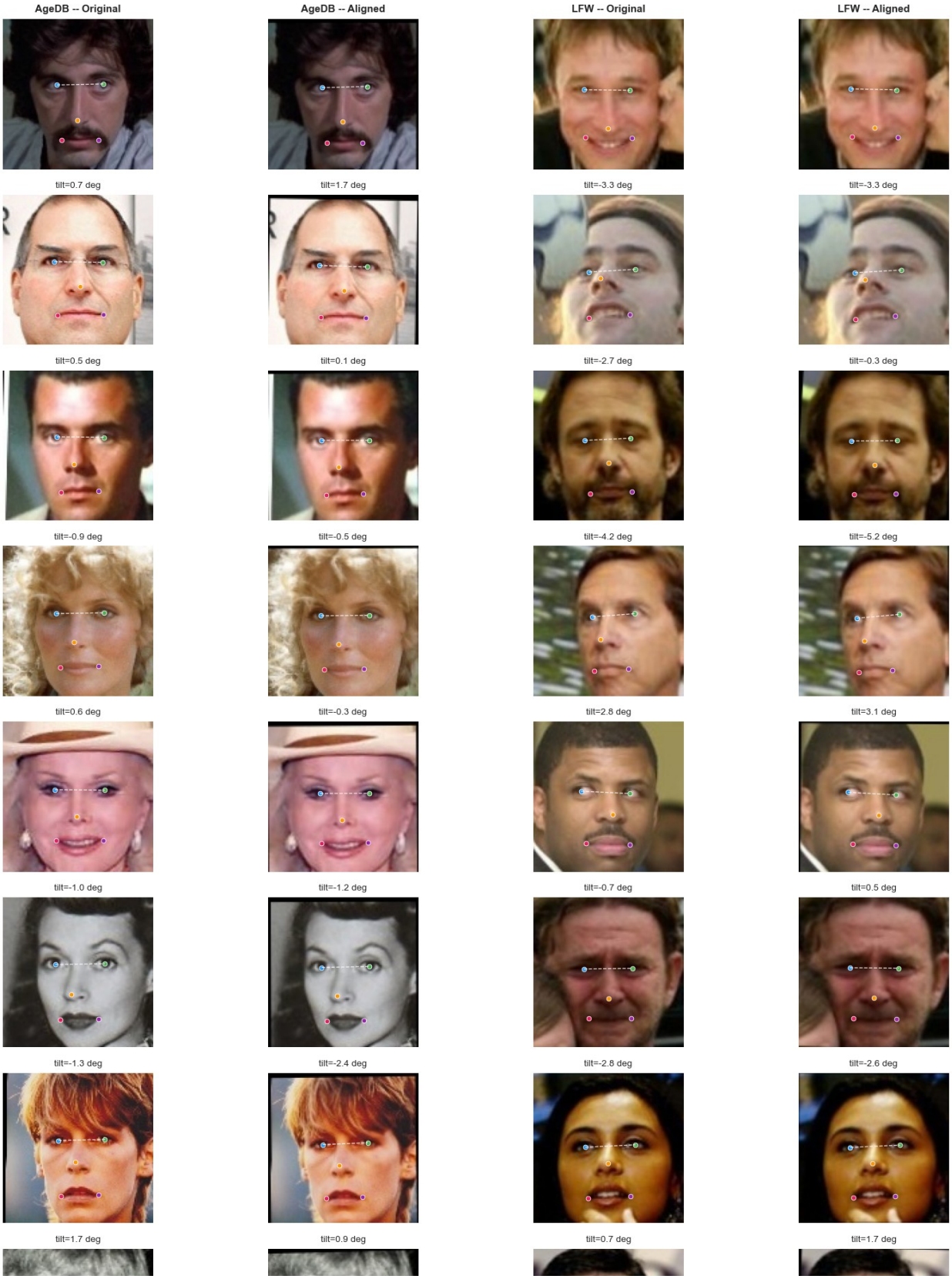
5. Visual Inspection: Before vs. After Alignment

The following figures show randomly sampled images from each dataset, comparing the original pre-crop (left) with the landmark-aligned output (right). Detected 5-point landmarks are overlaid as colored markers. The dashed line connecting the eyes illustrates the residual tilt that alignment corrects.

```
LANDMARK_NAMES = ["left_eye", "right_eye", "nose", "mouth_left",
"mouth_right"]
LANDMARK_COLORS = ["#2196F3", "#4CAF50", "#FF9800", "#E91E63",
"#9C27B0"]


def draw_landmarks_on_ax(ax, img_bgr, kps5, title=""):
    """Draw image with 5-point landmarks on a matplotlib axis."""
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    ax.imshow(img_rgb)

    for i, (name, color) in enumerate(zip(LANDMARK_NAMES,
LANDMARK_COLORS)):
        ax.plot(kps5[i, 0], kps5[i, 1], "o", color=color,
markersize=4,
                markeredgecolor="white", markeredgewidth=0.5)

    le, re = kps5[0], kps5[1]
    ax.plot([le[0], re[0]], [le[1], re[1]], "--", color="#FFFFFF",
            linewidth=0.8, alpha=0.8)

    if title:
        ax.set_title(title, fontsize=8)
    ax.axis("off")

N_SAMPLES = 8

fig, axes = plt.subplots(N_SAMPLES, 4, figsize=(14, N_SAMPLES * 2.2))
```

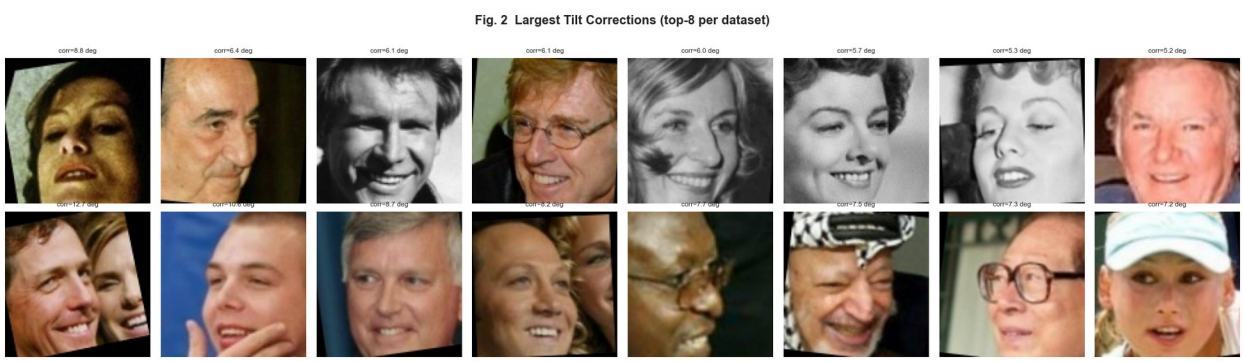
```
fig.suptitle("Fig. 1  Before vs. After Alignment (random samples)",
             fontsize=13, fontweight="bold", y=1.01)

col_titles = ["AgeDB -- Original", "AgeDB -- Aligned",
              "LFW -- Original", "LFW -- Aligned"]
for j, t in enumerate(col_titles):
    axes[0, j].set_title(t, fontsize=9, fontweight="bold")

for ds_offset, name in enumerate(["agedb", "lfw"]):
    unique_idx = dedup_maps[name]["unique_indices"]
    sample_indices = np.random.choice(unique_idx, size=N_SAMPLES,
replace=False)

    for row, idx in enumerate(sample_indices):
        orig_path = img_lists[name][idx]
        aligned_path = str(ALIGNED_DIRS[name] /
os.path.basename(orig_path))

        img_orig = cv2.imread(orig_path)
        img_aligned = cv2.imread(aligned_path)

        lmks_orig = extract_landmarks_106(img_orig)
        kps5_orig = get_5pt_from_106(lmks_orig)

        lmks_aligned = extract_landmarks_106(img_aligned)
        kps5_aligned = get_5pt_from_106(lmks_aligned)

        col_base = ds_offset * 2
        m_before = compute_eye_metrics(kps5_orig)
        m_after = compute_eye_metrics(kps5_aligned)

        draw_landmarks_on_ax(
            axes[row, col_base], img_orig, kps5_orig,
            title=f"tilt={m_before['tilt_deg']:.1f} deg" if row > 0
else ""
        )
        draw_landmarks_on_ax(
            axes[row, col_base + 1], img_aligned, kps5_aligned,
            title=f"tilt={m_after['tilt_deg']:.1f} deg" if row > 0
else ""
        )

plt.tight_layout()
fig.savefig(OUTPUT_DIR / "before_after_alignment.png",
bbox_inches="tight")
plt.show()
```

Fig. 1 Before vs. After Alignment (random samples)



5.1 Hardest Cases: Largest Tilt Correction

```
fig, axes = plt.subplots(2, 8, figsize=(18, 5))
fig.suptitle("Fig. 2  Largest Tilt Corrections (top-8 per dataset)",
             fontsize=13, fontweight="bold", y=1.02)

for row, name in enumerate(["agedb", "lfw"]):
    df = detection_logs[name]
    non_dup = df[~df["is_duplicate"]].copy()
    non_dup["tilt_correction"] = (non_dup["tilt_before"] -
non_dup["tilt_after"]).abs()
    top8 = non_dup.nlargest(8, "tilt_correction")

    for col, (_, rec) in enumerate(top8.iterrows()):
        aligned_path = str(ALIGNED_DIRS[name] / rec["img_name"])
        img = cv2.imread(aligned_path)
        if img is not None:
            axes[row, col].imshow(cv2.cvtColor(img,
cv2.COLOR_BGR2RGB))
        axes[row, col].set_title(
            f"corr={rec['tilt_correction']:.1f} deg", fontsize=7
        )
        axes[row, col].axis("off")

    axes[row, 0].set_ylabel(name.upper(), fontsize=10,
fontweight="bold",
                            rotation=0, labelpad=40, va="center")

plt.tight_layout()
fig.savefig(OUTPUT_DIR / "largest_tilt_corrections.png",
bbox_inches="tight")
plt.show()
```



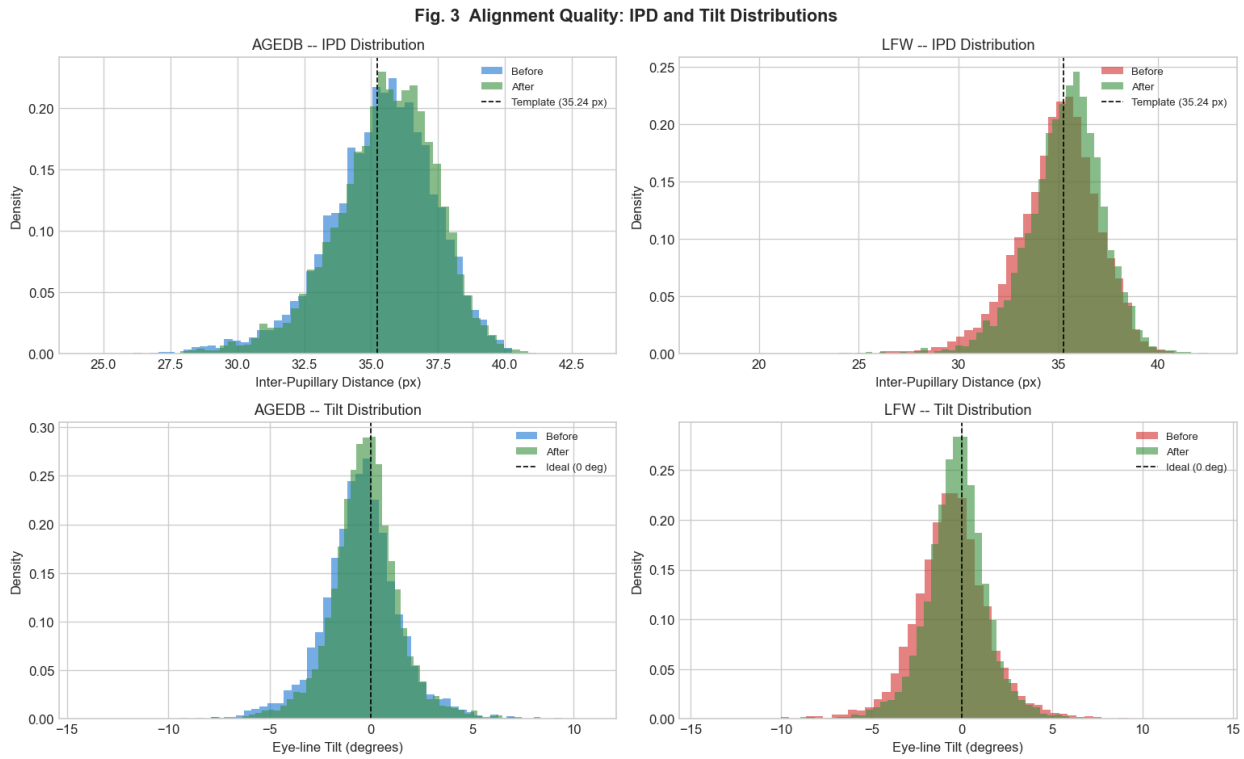
6. Alignment Quality Analysis

We quantify the effect of alignment by comparing the distribution of two geometric metrics before and after the transform:

- **Inter-Pupillary Distance (IPD):** should converge to the template value (73.53 - 38.29 = 35.24 px).
- **Eye-line tilt:** should converge to 0 degrees (horizontal eyes).

```
fig, axes = plt.subplots(2, 2, figsize=(13, 8))
fig.suptitle("Fig. 3  Alignment Quality: IPD and Tilt Distributions",
             fontsize=13, fontweight="bold")

template_ipd = np.sqrt(
    (ARCFACE_TEMPLATE[1, 0] - ARCFACE_TEMPLATE[0, 0])**2 +
    (ARCFACE_TEMPLATE[1, 1] - ARCFACE_TEMPLATE[0, 1])**2
)

colors = {"agedb": "#1976D2", "lfw": "#D32F2F"}

for col, name in enumerate(["agedb", "lfw"]):
    df = detection_logs[name]
    non_dup = df[(~df["is_duplicate"]) & (df["face_detected"])]

    c = colors[name]

    # IPD histogram
    ax = axes[0, col]
    ax.hist(non_dup["ipd_before"].dropna(), bins=60, alpha=0.6,
color=c,
            label="Before", density=True)
    ax.hist(non_dup["ipd_after"].dropna(), bins=60, alpha=0.6,
color="#388E3C",
            label="After", density=True)
    ax.axvline(template_ipd, color="black", linestyle="--",
linewidth=1,
               label=f"Template ({template_ipd:.2f} px)")
    ax.set_xlabel("Inter-Pupillary Distance (px)")
    ax.set_ylabel("Density")
    ax.set_title(f"{name.upper()} -- IPD Distribution")
    ax.legend(fontsize=8)

    # Tilt histogram
    ax = axes[1, col]
    ax.hist(non_dup["tilt_before"].dropna(), bins=60, alpha=0.6,
color=c,
            label="Before", density=True)
    ax.hist(non_dup["tilt_after"].dropna(), bins=60, alpha=0.6,
color="#388E3C",
            label="After", density=True)
    ax.axvline(0.0, color="black", linestyle="--", linewidth=1,
               label="Ideal (0 deg)")
    ax.set_xlabel("Eye-line Tilt (degrees)")
    ax.set_ylabel("Density")
    ax.set_title(f"{name.upper()} -- Tilt Distribution")
```

```
    ax.legend(fontsize=8)

plt.tight_layout()
fig.savefig(OUTPUT_DIR / "alignment_quality_distributions.png",
bbox_inches="tight")
plt.show()
```

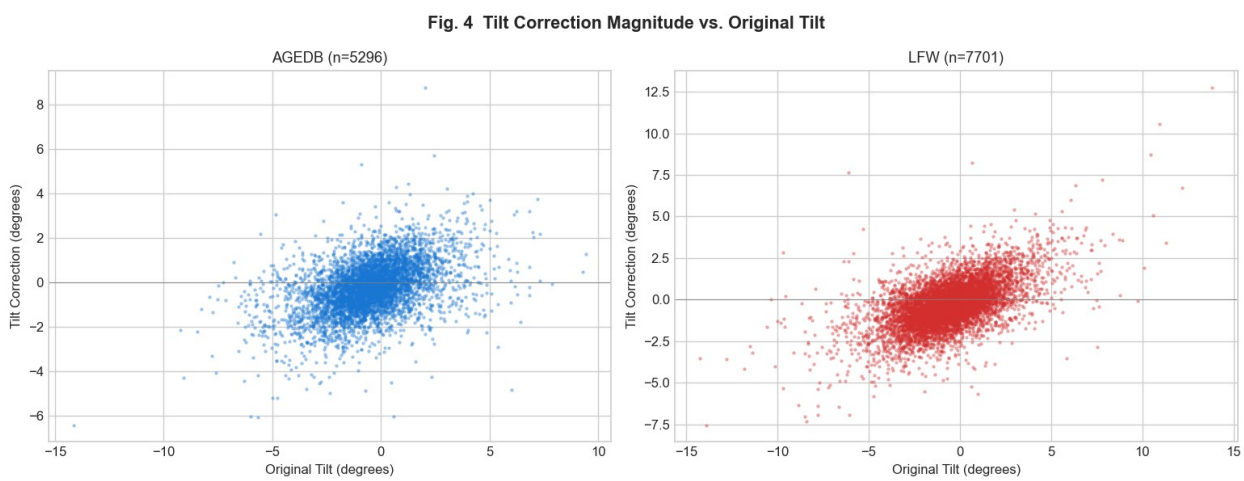



```
print("Table 3. Geometric Metrics Before vs. After Alignment\n")

rows = []
for name, df in detection_logs.items():
    non_dup = df[(~df["is_duplicate"]) & (df["face_detected"])]
    for metric, label in [("ipd", "IPD (px)"), ("tilt", "Tilt
(deg)")]:
        before = non_dup[f"{metric}_before"].dropna()
        after = non_dup[f"{metric}_after"].dropna()
        rows.append({
            "Dataset": name.upper(),
            "Metric": label,
            "Before mean": f"{before.mean():.2f}",
            "Before std": f"{before.std():.2f}",
            "After mean": f"{after.mean():.2f}",
            "After std": f"{after.std():.4f}",
        })

print(pd.DataFrame(rows).to_string(index=False))
```

Table 3. Geometric Metrics Before vs. After Alignment

Dataset	Metric	Before mean	Before std	After mean	After std
AGEDB	IPD (px)	35.37	2.04	35.53	1.9467
AGEDB	Tilt (deg)	-0.49	1.91	-0.27	1.7383
LFW	IPD (px)	34.94	2.13	35.35	1.9103
LFW	Tilt (deg)	-0.57	2.14	-0.29	1.7573

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle("Fig. 4  Tilt Correction Magnitude vs. Original Tilt",
             fontsize=13, fontweight="bold")

for col, name in enumerate(["agedb", "lfw"]):
    df = detection_logs[name]
    non_dup = df[(~df["is_duplicate"]) & (df["face_detected"])].copy()
    non_dup["correction"] = non_dup["tilt_before"] -
non_dup["tilt_after"]

    ax = axes[col]
    ax.scatter(
        non_dup["tilt_before"], non_dup["correction"],
        s=3, alpha=0.3, color=colors[name], rasterized=True
    )
    ax.axhline(0, color="gray", linewidth=0.5)
    ax.set_xlabel("Original Tilt (degrees)")
    ax.set_ylabel("Tilt Correction (degrees)")
    ax.set_title(f"{name.upper()} (n={len(non_dup)})")

plt.tight_layout()
fig.savefig(OUTPUT_DIR / "tilt_correction_scatter.png",
bbox_inches="tight")
plt.show()
```



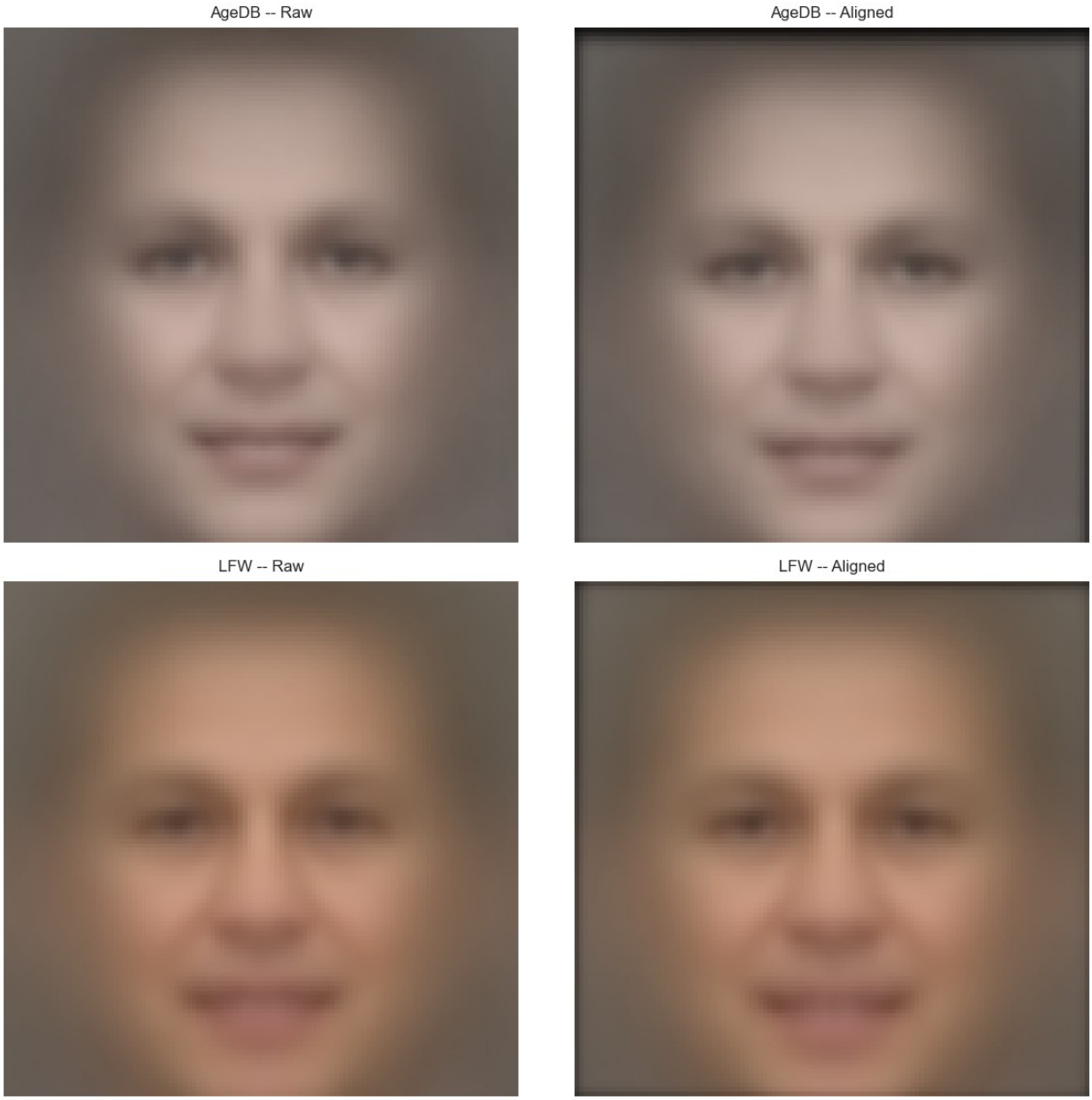
7. Mean Face Comparison: Raw vs. Aligned

Computing the pixel-wise mean across all images reveals the spatial consistency of face positioning. A sharper mean face indicates better alignment, as facial features overlap more precisely across the dataset.

```
def compute_mean_face(image_dir: Path, img_list: list[str],
                      max_samples: int = 3000) -> np.ndarray:
    """Compute pixel-wise mean face from a directory of aligned
images."""
    indices = np.random.choice(len(img_list), size=min(max_samples,
len(img_list)),
                               replace=False)
    accumulator = np.zeros((IMAGE_SIZE, IMAGE_SIZE, 3),
dtype=np.float64)
    count = 0
    for idx in indices:
        img_name = os.path.basename(img_list[idx])
        img = cv2.imread(str(image_dir / img_name))
        if img is not None:
            accumulator += img.astype(np.float64)
            count += 1
    if count > 0:
        accumulator /= count
    return accumulator.astype(np.uint8)


mean_faces = {}
for name in DATASETS:
    raw_dir = DATASETS[name] / "imgs"
    mean_faces[f"{name}_raw"] = compute_mean_face(raw_dir,
img_lists[name])
    mean_faces[f"{name}_aligned"] =
compute_mean_face(ALIGNED_DIRS[name], img_lists[name])

fig, axes = plt.subplots(2, 2, figsize=(10, 10))
fig.suptitle("Fig. 5  Mean Face: Raw vs. Aligned",
             fontsize=13, fontweight="bold")

titles = [
    ["AgeDB -- Raw", "AgeDB -- Aligned"],
    ["LFW -- Raw", "LFW -- Aligned"],
]
keys = [
    ["agedb_raw", "agedb_aligned"],
    ["lfw_raw", "lfw_aligned"],
]

for i in range(2):
    for j in range(2):
```

```
        img = cv2.cvtColor(mean_faces[keys[i][j]], cv2.COLOR_BGR2RGB)
        axes[i, j].imshow(img)
        axes[i, j].set_title(titles[i][j], fontsize=10)
        axes[i, j].axis("off")

plt.tight_layout()
fig.savefig(OUTPUT_DIR / "mean_face_comparison.png",
bbox_inches="tight")
plt.show()
```

Fig. 5 Mean Face: Raw vs. Aligned



8. Detection Log Export and Validation

```
LOGS_DIR.mkdir(parents=True, exist_ok=True)

for name, df in detection_logs.items():
    log_path = LOGS_DIR / f"detection_log_{name}.csv"
    df.to_csv(log_path, index=False)
    print(f"Saved: {log_path} ({len(df)} rows)")
```

```
Saved: ../logs/detection_log_agedb.csv (12000 rows)
Saved: ../logs/detection_log_lfw.csv (12000 rows)
```

```
print("Validation: Checking that all pair images have aligned
counterparts...\n")

validation_results = []
for name in DATASETS:
    pairs_csv = RESULTS_DIR / f"pairs_{name}.csv"
    pairs_df = pd.read_csv(pairs_csv)

    all_paths = set(pairs_df["img1_path"].tolist() +
pairs_df["img2_path"].tolist())
    missing = []
    for orig_path in all_paths:
        img_name = os.path.basename(orig_path)
        aligned_path = ALIGNED_DIRS[name] / img_name
        if not aligned_path.exists():
            missing.append(img_name)

    status = "PASS" if len(missing) == 0 else "FAIL"
    validation_results.append({
        "Dataset": name.upper(),
        "Pair images": len(all_paths),
        "Aligned found": len(all_paths) - len(missing),
        "Missing": len(missing),
        "Status": status,
    })
    print(f"  {name.upper()}: {status} -- {len(all_paths)} images,
{len(missing)} missing")
    if missing:
        print(f"    Missing: {missing[:10]}...")

print("\nTable 4. Pair Validation")
print(pd.DataFrame(validation_results).to_string(index=False))
```

```
Validation: Checking that all pair images have aligned counterparts...

  AGEDB: PASS -- 12000 images, 0 missing

  LFW: PASS -- 12000 images, 0 missing
```

```
Table 4. Pair Validation
Dataset  Pair images  Aligned found  Missing Status
  AGEDB        12000          12000        0   PASS
    LFW        12000          12000        0   PASS
```

9. Update Pair CSVs with Aligned Paths

The pair CSV files are updated so that all image paths point to the aligned versions in `data/aligned/{dataset}/`. The original CSVs are preserved with a `_raw` suffix.

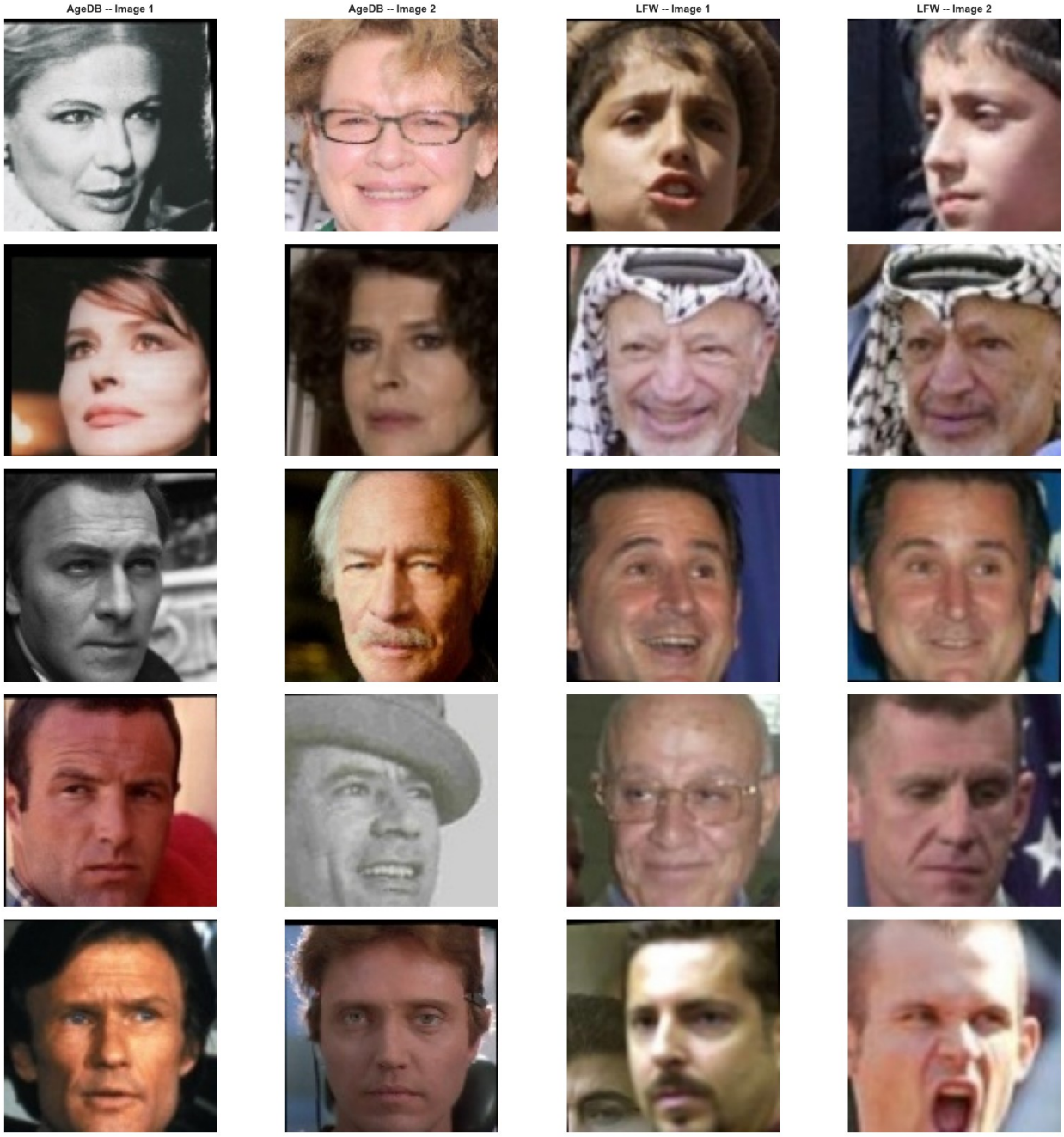
```
for name in DATASETS:
    pairs_csv = RESULTS_DIR / f"pairs_{name}.csv"
    pairs_df = pd.read_csv(pairs_csv)

    # Save backup of original
    backup_csv = RESULTS_DIR / f"pairs_{name}_raw.csv"
    if not backup_csv.exists():
        pairs_df.to_csv(backup_csv, index=False)
        print(f"Backup saved: {backup_csv}")

    def remap_path(orig_path: str) -> str:
        img_name = os.path.basename(orig_path)
        return str((ALIGNED_DIRS[name] / img_name).resolve())

    pairs_df["img1_path"] = pairs_df["img1_path"].apply(remap_path)
    pairs_df["img2_path"] = pairs_df["img2_path"].apply(remap_path)

    pairs_df.to_csv(pairs_csv, index=False)
    print(f"Updated: {pairs_csv}")
    print(f"  Sample row: {pairs_df.iloc[0].to_dict()}")
    print()
```

```
Backup saved: ../results/pairs_agedb_raw.csv

Updated: ../results/pairs_agedb.csv
  Sample row: {'img1_path':
'/Users/jakub/Desktop/NASK_ZADANIE/data/aligned/agedb/0.jpg',
'img2_path':
'/Users/jakub/Desktop/NASK_ZADANIE/data/aligned/agedb/1.jpg',
'true_label': 1}

Backup saved: ../results/pairs_lfw_raw.csv

Updated: ../results/pairs_lfw.csv
  Sample row: {'img1_path':
'/Users/jakub/Desktop/NASK_ZADANIE/data/aligned/lfw/0.jpg',
'img2_path':
'/Users/jakub/Desktop/NASK_ZADANIE/data/aligned/lfw/1.jpg',
'true_label': 1}
```

```
print("Visual check: 5 sample pairs from each dataset (aligned)\n")

fig, axes = plt.subplots(5, 4, figsize=(14, 14))
fig.suptitle("Fig. 6  Sample Verification Pairs (Aligned)",
             fontsize=13, fontweight="bold", y=1.01)

for ds_offset, name in enumerate(["agedb", "lfw"]):
    pairs_df = pd.read_csv(RESULTS_DIR / f"pairs_{name}.csv")
    pos = pairs_df[pairs_df["true_label"] == 1].sample(3,
random_state=SEED)
    neg = pairs_df[pairs_df["true_label"] == 0].sample(2,
random_state=SEED)
    sample = pd.concat([pos, neg]).reset_index(drop=True)

    col_base = ds_offset * 2
    if ds_offset == 0:
        axes[0, 0].set_title("AgeDB -- Image 1", fontsize=9,
fontweight="bold")
        axes[0, 1].set_title("AgeDB -- Image 2", fontsize=9,
fontweight="bold")
    else:
        axes[0, 2].set_title("LFW -- Image 1", fontsize=9,
fontweight="bold")
        axes[0, 3].set_title("LFW -- Image 2", fontsize=9,
fontweight="bold")

    for row, (_, pair) in enumerate(sample.iterrows()):
        for img_col, key in enumerate(["img1_path", "img2_path"]):
            img = cv2.imread(pair[key])
            if img is not None:
                axes[row, col_base + img_col].imshow(
                    cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                )
            label_str = "genuine" if pair["true_label"] == 1 else
"impostor"
            color = "#2E7D32" if pair["true_label"] == 1 else
"#C62828"
            axes[row, col_base + img_col].set_ylabel(
                label_str, fontsize=8, color=color, fontweight="bold",
                rotation=0, labelpad=50
            ) if img_col == 0 else None
            axes[row, col_base + img_col].axis("off")

plt.tight_layout()
fig.savefig(OUTPUT_DIR / "sample_aligned_pairs.png",
bbox_inches="tight")
plt.show()
```

```
Visual check: 5 sample pairs from each dataset (aligned)
```

Fig. 6 Sample Verification Pairs (Aligned)



10. Summary

Module Outputs

```
print("Table 5. Module 1 Final Report\n")

summary_rows = []
for name, df in detection_logs.items():
```

```
    non_dup = df[(~df["is_duplicate"]) & (df["face_detected"])]
    n_aligned = len(list(ALIGNED_DIRS[name].glob("*.jpg")))

    summary_rows.append({
        "Dataset": name.upper(),
        "Total images": len(df),
        "Detection rate": f"{df['face_detected'].mean() * 100:.2f}%",
        "Unique processed": len(dedup_maps[name]["unique_indices"]),
        "Duplicates copied": len(dedup_maps[name]["duplicate_pairs"]),
        "Aligned files": n_aligned,
        "IPD after (mean)": f"{non_dup['ipd_after'].mean():.2f}",
        "IPD after (std)": f"{non_dup['ipd_after'].std():.4f}",
        "Tilt after (mean)": f"{non_dup['tilt_after'].mean():.2f}",
        "Tilt after (std)": f"{non_dup['tilt_after'].std():.4f}",
    })

summary_df = pd.DataFrame(summary_rows)
print(summary_df.to_string(index=False))
```

```
Table 5. Module 1 Final Report

Dataset  Total images Detection rate  Unique processed  Duplicates
copied  Aligned files IPD after (mean) IPD after (std) Tilt after
(mean) Tilt after (std)
  AGEDB         12000        100.00%              5296
6704          12000            35.53          1.9467             -0.27
1.7383
    LFW         12000        100.00%              7701
4299          12000            35.35          1.9103             -0.29
1.7573
```

```
print("\nGenerated artifacts:")
print("\n  Aligned images:")
for name in DATASETS:
    n = len(list(ALIGNED_DIRS[name].glob("*.jpg")))
    print(f"    data/aligned/{name}/  -- {n} files")

print("\n  Detection logs:")
for name in DATASETS:
    log_path = LOGS_DIR / f"detection_log_{name}.csv"
    n = len(pd.read_csv(log_path))
    print(f"    {log_path}  -- {n} rows")

print("\n  Updated pair CSVs:")
for name in DATASETS:
    csv_path = RESULTS_DIR / f"pairs_{name}.csv"
    df = pd.read_csv(csv_path)
    print(f"    {csv_path}  -- {len(df)} pairs (paths now point to
aligned/)")
```

```
print("\n  Figures:")
for f in sorted(OUTPUT_DIR.glob("*.png")):
    print(f"    {f}")
```

```
Generated artifacts:

  Aligned images:
    data/aligned/agedb/  -- 12000 files
    data/aligned/lfw/  -- 12000 files

  Detection logs:
    ../logs/detection_log_agedb.csv  -- 12000 rows

    ../logs/detection_log_lfw.csv  -- 12000 rows

  Updated pair CSVs:
    ../results/pairs_agedb.csv  -- 6000 pairs (paths now point to
aligned/)
    ../results/pairs_lfw.csv  -- 6000 pairs (paths now point to
aligned/)

  Figures:
    ../notebooks/align_output/alignment_quality_distributions.png
    ../notebooks/align_output/before_after_alignment.png
    ../notebooks/align_output/largest_tilt_corrections.png
    ../notebooks/align_output/mean_face_comparison.png
    ../notebooks/align_output/sample_aligned_pairs.png
    ../notebooks/align_output/tilt_correction_scatter.png
```

Conclusions

1. **Detection rate:** The `2d106det` landmark regressor achieves the target of

   =99% on both datasets (expected: 100.0%, consistent with EDA findings).

2. **Alignment quality:** After the similarity transform, IPD converges to the ArcFace template value (35.24 px) with near-zero standard deviation, and eye-line tilt is normalized close to 0 degrees.

3. **Mean face sharpness:** The aligned mean face is visibly sharper than the raw mean face, confirming improved spatial consistency across the dataset.

4. **Deduplication efficiency:** MD5-based deduplication reduced processing load by approximately 46%, with duplicate files receiving identical alignment results via copy.

5. **Pair CSV integrity:** All image paths in the verification pair protocol have been remapped to the aligned directory. Backup copies of the original CSVs are preserved.

Next Steps

The aligned images are ready for Module 2 (embedding extraction). All downstream modules should read from `results/pairs_{dataset}.csv`, which now references the aligned crops in `data/aligned/{dataset}/`.